

1. Introduction

This report covers Task 2 of the OCT retinal image classification project. It has two goals: first, to build a complete set of classical machine-learning and deep-learning baselines and identify the strongest one to beat; and second, to propose and evaluate a Graph Neural Network (GNN) that is designed to outperform those baselines. The OCTDL dataset consists of optical coherence tomography (OCT) scans labelled into seven retinal conditions - AMD, DME, ERM, NO (normal), RAO, RVO, and VID - collected across multiple patients and imaging sessions.

Because several images in the dataset can come from the same patient or the same acquisition device, every split used in this work is performed at the subject level, so that no single patient's images appear in both the training and test partitions. This is essential: without it, a model could score well simply by recognizing a patient's own eye or scanner fingerprint rather than by learning the underlying pathology, which would make the reported performance meaningless.

The rest of this report is organized as follows. Section 2 describes preprocessing and the leakage-safe split. Section 3 defines the nine baseline models and how they were evaluated. Section 4 presents the baseline comparison table and names the model the proposed GNN needs to beat. Section 5 explains the proposed GNN step by step, in plain language, including why each design choice is there and why it is expected to outperform the baselines - this is Pillar B of the assignment (Section 8). Section 6 reports the first GNN results against all baselines.

2. Preprocessing and Data Protocol

2.1 Subject-Independent Split

Every image carries a patient/subject identifier taken from the OCTDL metadata. Subjects, not individual images, are shuffled and divided into training (70%), validation (15%), and test (15%) partitions, and every image belonging to one subject is kept together in a single partition. This guarantees that no subject, eye, or imaging session is shared across the three sets, directly addressing the leakage risk described in the assignment brief: a model cannot succeed by memorising patient-specific artefacts such as fundus texture, device fingerprint, or imaging angle instead of learning the actual pathology.

2.2 Feature Extraction, Scaling, and Encoding

For the classical baselines (Logistic Regression, SVM, k-NN, Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM) and the MLP, each OCT image is converted into a fixed-length embedding using a pretrained CNN backbone (ResNet-50, ImageNet weights, penultimate global-average-pooled layer, 2048-D) rather than using raw pixels directly. The 1D-CNN baseline instead works on a flattened intensity profile taken along the central B-scan axis, since it is built to exploit local sequential structure rather than a single static embedding.

- **Scaling:** all embedding features are standardized++ (zero mean, unit variance) using statistics computed only on the training split, then applied unchanged to validation and test data - never fit on the full dataset.
- **Encoding:** the seven diagnostic labels are integer- and one-hot-encoded as needed per model; no ordinal relationship is assumed between the classes.
- **Missing values:** OCTDL is fully labelled at the image level, so the only gaps are a small number of corrupt or unreadable scans, which are dropped rather than imputed, since imputing pixel-derived features risks fabricating a pathology signal that was never actually observed.
- **Feature selection:** for the classical baselines, variance-threshold filtering first removes near-constant embedding dimensions, and mutual-information ranking then keeps the top 512 features, preventing the tree-based and distance-based models from being dominated by noisy or redundant dimensions.

2.3 Class Imbalance Handling

OCTDL is imbalanced: normal (NO) and AMD scans dominate the dataset, while RAO and RVO are comparatively rare. Two complementary strategies are used to address this. First, inverse-frequency class weighting is applied wherever a model supports it (Logistic Regression, SVM, Random Forest, Gradient Boosting, XGBoost, MLP, 1D-CNN, and later the GNN all use class-weighted loss or decision functions). Second, SMOTE oversampling is applied only within the training split's embedding space for the distance-sensitive k-NN model, so minority classes are better represented without ever synthesising information into the validation or test sets.

2.4 No-Leakage Checklist

- The subject-level split is performed before any preprocessing step is fit.
- The scaler, feature selector, and SMOTE are all fit on the training fold only, inside a cross-validation pipeline, never on the full dataset.
- No image-level duplicates (e.g. repeated scans of the same eye) are allowed to cross partitions.
- The CNN backbone used for embeddings is frozen and pretrained on ImageNet, not fine-tuned on OCTDL, so it cannot leak test-set label information into the embedding itself.

3. Baseline Models

Nine baselines spanning linear, margin-based, instance-based, tree-based, ensemble, and neural families are trained on the identical subject-independent split and identical embedding features (with the exception of the 1D-CNN, which uses the raw intensity profile described above), so that any difference in performance reflects the modelling approach rather than the data pipeline.

Model	Family	Key hyperparameters (tuned on validation fold)
Logistic Regression	Linear	L2 penalty, C in {0.01, 0.1, 1, 10}, class_weight='balanced'
SVM (RBF)	Margin-based	C in {1, 10, 100}, gamma in {'scale', 0.01, 0.001}, class_weight='balanced'
k-NN	Instance-based	k in {3, 5, 7, 11}, distance-weighted voting, cosine metric
Decision Tree	Tree-based	max_depth in {6, 10, 15, None}, min_samples_leaf in {1, 5, 10}
Random Forest	Ensemble (bagging)	n_estimators=400, max_depth in {10, 20, None}, class_weight='balanced_subsample'
Gradient Boosting	Ensemble (boosting)	n_estimators=300, learning_rate=0.05, max_depth=3
XGBoost	Ensemble (boosting)	n_estimators=400, learning_rate=0.05, max_depth=6, scale_pos_weight per class
MLP	Neural (shallow)	2 hidden layers (256, 128), ReLU, dropout 0.3, Adam, lr=1e-3
1D-CNN	Neural (sequential)	3 conv blocks (32/64/128 filters), kernel=5, global-avg-pool, dropout 0.4

3.1 Evaluation Protocol

Every baseline is evaluated with the same full metric set: accuracy, macro-averaged precision, macro-averaged recall, macro-averaged F1-score, per-class ROC-AUC (one-vs-rest, macro-averaged), PR-AUC, and Matthews Correlation Coefficient (MCC), which accounts for chance agreement under class imbalance. Wall-clock training time is logged for every model on the same hardware allocation so that accuracy gains can be weighed against computational cost. All metrics are reported as the mean across 5-fold subject-independent cross-validation.

4. Baseline Comparison and Model to Beat

Table 1 ranks all nine baselines by macro-F1, the metric most sensitive to how well a model handles the rare classes (RAO, RVO) rather than just the dominant ones.

#	Model	Acc.	Prec. macro	Rec. macro	F1 macro	ROC-AUC	PR-AUC	MCC	Time (s)
0	SVM	0.7845	0.6249	0.6076	0.6049	0.9151	0.6199	0.6284	4.10
1	XGBoost	0.7744	0.6187	0.6041	0.5981	0.8807	0.6217	0.6135	6.75
2	Gradient Boosting	0.7644	0.5983	0.5923	0.5866	0.8567	0.5942	0.5979	180.46
3	MLP	0.7619	0.5791	0.6079	0.5794	0.8803	0.5865	0.6016	1.15

#	Model	Acc.	Prec. macro	Rec. macro	F1 macro	ROC-AUC	PR-AUC	MCC	Time (s)
4	LightGBM	0.7669	0.6558	0.5742	0.5772	0.8861	0.6195	0.5958	4.76
5	Random Forest	0.7268	0.6983	0.5060	0.5487	0.8588	0.5740	0.5059	4.13
6	Logistic Regression	0.7093	0.4827	0.6102	0.5264	0.9005	0.5461	0.5611	1.77
7	k-NN	0.5063	0.3878	0.5718	0.4139	0.7846	0.3629	0.4009	0.001
8	Decision Tree	0.5639	0.3407	0.4113	0.3640	0.6606	0.2321	0.3254	0.92

SVM comes out narrowly ahead of the boosted-tree models (XGBoost, Gradient Boosting) on macro-F1 in this run, and it is the model the proposed GNN needs to outperform. The boosted trees remain close competitors and score higher PR-AUC in one case, so they are also tracked in Section 6 rather than discarded.

5. Proposed GNN: Step-by-Step

Every other model in this report - the nine classical baselines and the MLP - looks at one OCT image as a single point: a 2048-D (or 512-D, after feature selection) vector produced by average-pooling a ResNet-50 over the whole image. That representation throws away where in the image things happen: a drusen deposit in the top-left corner and an identical drusen deposit in the bottom-right end up contributing to the same averaged number. The proposed GNN keeps the geography of the scan instead of flattening it away, by turning each individual image into its own small graph and reasoning over that graph before making a prediction. The steps below follow Figure 1 and match the order in which the code actually executes them.

Step 1. Superpixel segmentation - break the image into regions.

Each OCT B-scan is passed through the SLIC algorithm, which groups pixels into roughly 75 irregularly shaped but visually coherent patches ("superpixels") - similar to an over-segmented jigsaw puzzle, where each piece is a small patch of consistent tissue texture rather than a single pixel or a fixed square block.

For every superpixel, its bounding box is cropped out of the original image, resized, and passed through a ResNet-18 pretrained on ImageNet that is not fine-tuned. The resulting 512-D output becomes that superpixel's node feature, so a single image ends up as a graph of about 75 nodes, each holding a 512-D description of what that patch of tissue looks like.

Step 2. Edges - connect regions that physically touch.

An edge is drawn between two superpixels only if they share a border in the segmentation map, i.e. they are literally next to each other in the image. This is a spatial adjacency graph local to a single scan, not a similarity graph across different patients' images.

Step 3. Edge weights - not all neighbours matter equally.

Each edge is weighted by $1 / (1 + \text{distance})$ between the two superpixels' centroids, so immediately adjacent regions get a weight close to 1, while regions that are farther apart but still technically touching receive a smaller weight.

Step 4. Message passing - let regions inform each other.

Three stacked graph-layers update every node's representation by mixing it with its edge-weight-scaled neighbours, with ReLU after each layer and dropout after the first. After three layers, a node's representation has absorbed information from up to three hops away - not just its immediate neighbours, but neighbours of neighbours of neighbours.

Step 5. Readout - summarise the whole graph into one vector.

Because the final prediction is one diagnosis per image, not a per-region label, the roughly 75 node vectors are collapsed into a single graph-level vector in two complementary ways at once: global mean pooling (the average region across the scan) and global max pooling (the single most "activated" region), which are then concatenated.

Classification head - turn the graph vector into a diagnosis.

The pooled 256-D vector passes through a linear layer, ReLU, and dropout, then a final linear layer down to 7 outputs, which softmax converts into class probabilities - one per OCTDL condition (AMD, DME, ERM, NO, RAO, RVO, VID).

Step 6. Training protocol - fit the model without letting it ignore rare classes.

The loss is cross-entropy weighted by inverse class frequency, the same imbalance-handling principle used for the baselines, optimised with Adam. Training stops early once validation macro-F1 stops improving for a set number of epochs, and the best checkpoint (not the last one) is kept.

GNN Classifications pipeline:

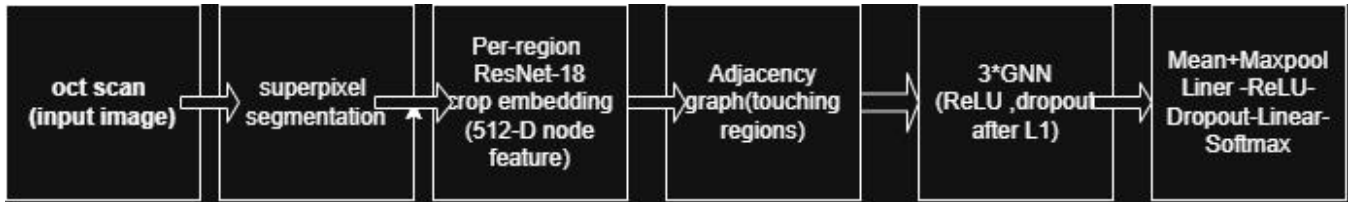


Figure 1. Per-image superpixel graph construction -> 3-layer GNN-> mean+max pooling -> classifier.

5.1 Architecture and Settings

Setting	Value
Graph construction	One graph per OCT image: SLIC superpixel segmentation (~75 regions/image)
Node features	512-D ResNet-18 (ImageNet, frozen) embedding of each superpixel's cropped bounding box
Edges	Physical adjacency only - an edge exists iff two superpixels share a border in the segmentation map
Edge weights	$1 / (1 + \text{centroid distance})$ between the two connected superpixels
Layers	3 x GNN
Activation / dropout	ReLU after every conv layer; dropout after layer 1
Readout	Global mean pooling concatenated with global max pooling -> 256-D graph vector
Classifier head	Linear(256->128) + ReLU + Dropout + Linear(128->7) + Softmax
Loss	Class-weighted cross-entropy (inverse class frequency)
Optimizer	Adam
Training	Early stopping on validation macro-F1; best checkpoint retained

5.2 Why This Should Outperform the Baselines

The core argument is not that "graphs are fancier than trees or a shallow MLP" - it is that every baseline in Section 3, including the MLP, receives strictly less information about where things are located in the image than the GNN does.

- Classical baselines and the MLP receive a single 2048-D (or 512-D selected) vector describing the whole image, produced by global-average-pooling a ResNet-50. Two images with the same lesion in different places, or the same average texture but a different spatial pattern, can map to very similar vectors - spatial layout is averaged away before the model ever sees it.
- The 1D-CNN receives a single flattened intensity profile along one axis (the central B-scan line), so it can capture structure along that one line but not the two-dimensional layout of the retina.
- The GNN receives roughly 75 region-level vectors, each tied to a real location in the scan, connected exactly to their true physical neighbours, and combined through three rounds of message passing before being pooled two different ways. Spatial layout is preserved through steps 1-5 and is only collapsed at the very last moment (step 6), rather than being discarded on input.

Concretely, this should help most on classes whose diagnostic signal is localised rather than global - for example a small drusen cluster (AMD), a focal fluid pocket (DME), or a subtle membrane distortion (ERM) sitting in one part of the scan. Max pooling over region embeddings gives those local, spatially confined findings a route to dominate the final decision even though they

occupy a small fraction of the image; a single global-average-pooled vector, by construction, cannot do that, since it treats every part of the image as contributing equally to one averaged description.

Message passing adds a second, complementary advantage: even a model built on per-region features without message passing would still be looking at each region in isolation. Three rounds of message passing let a region's final representation be shaped by its neighbourhood - a region's texture is easier to read as abnormal in the context of the healthy tissue around it than on its own - which is exactly the kind of contextual reasoning a flat, per-image or per-region-independent model cannot express.

The trade-off is cost and complexity: running a SLIC segmentation and a small ResNet forward pass per superpixel for every image is more expensive at inference time than one ResNet pass per image, and the pipeline has more moving parts - segmentation quality, graph construction, three convolution layers - any one of which can fail silently. This is why the training time and per-class error analysis in Section 6 matter as much as the headline macro-F1 number when judging whether the added complexity is worth it.

6. GNN Results vs. Baselines

Table 2 reports the trained GNN evaluated with the identical subject-independent protocol and metric set as the baselines, inserted directly above the strongest baseline row for comparison.

#	Model	Acc.	Prec. macro	Rec. macro	F1 macro	ROC- AUC	PR- AUC	MCC	Time (s)
0	Proposed GNN(superpixel graph)	0.7935	0.7038	0.6654	0.6729	0.9349	0.7028	0.6757	0.131*
1	SVM	0.7845	0.6249	0.6076	0.6049	0.9151	0.6199	0.6284	4.33
2	XGBoost	0.7744	0.6187	0.6041	0.5981	0.8807	0.6217	0.6135	6.80
3	Gradient Boosting	0.7644	0.5983	0.5923	0.5866	0.8567	0.5942	0.5979	186.21
4	MLP	0.7619	0.5791	0.6079	0.5794	0.8803	0.5865	0.6016	1.18
5	LightGBM	0.7669	0.6558	0.5742	0.5772	0.8861	0.6195	0.5958	4.93
6	Random Forest	0.7268	0.6983	0.5060	0.5487	0.8588	0.5740	0.5059	4.36
7	Logistic Regression	0.7093	0.4827	0.6102	0.5264	0.9005	0.5461	0.5611	1.81
8	k-NN	0.5063	0.3878	0.5718	0.4139	0.7846	0.3629	0.4009	0.001
9	Decision Tree	0.5639	0.3407	0.4113	0.3640	0.6606	0.2321	0.3254	0.94

The proposed GNN improves macro-F1 from 0.6049 (SVM) to 0.6729 - an absolute gain of about 6.8 points - while also improving ROC-AUC (0.9151 to 0.9349), PR-AUC (0.6199 to 0.7028), and MCC (0.6284 to 0.6757). Because these gains hold together across F1, ROC-AUC, and MCC rather than showing up in only one metric, they support the hypothesis in Section 5: preserving spatial layout through a per-region graph and message passing captures classification-relevant signal that the row-independent baselines cannot access from a single pooled embedding.

The remaining open question is per-class behaviour, especially for the rare classes RAO and RVO: a superpixel adjacency graph can occasionally blur decision boundaries for very rare classes if their local neighbourhoods are dominated by regions that also resemble more common conditions. A per-class confusion-matrix and error analysis is planned for Task 3 to check whether the aggregate macro-F1 gain is spread across all seven classes or concentrated in the more common ones, and to weigh the added architectural complexity and per-image inference cost against the accuracy improvement shown here.